LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Pirlo Syabila Hafuza (2509106008)

Kelas (A1 ‘25)

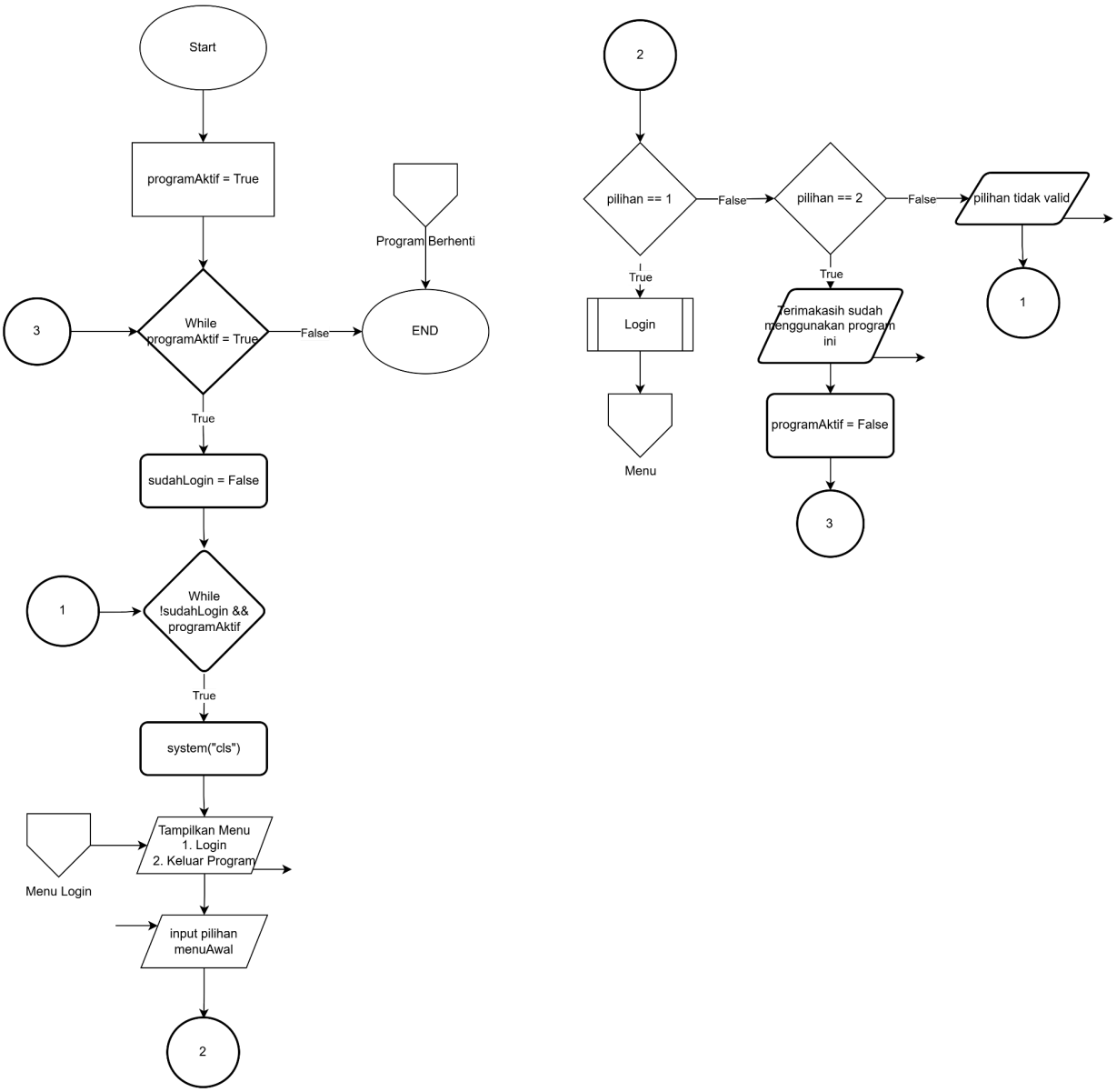
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

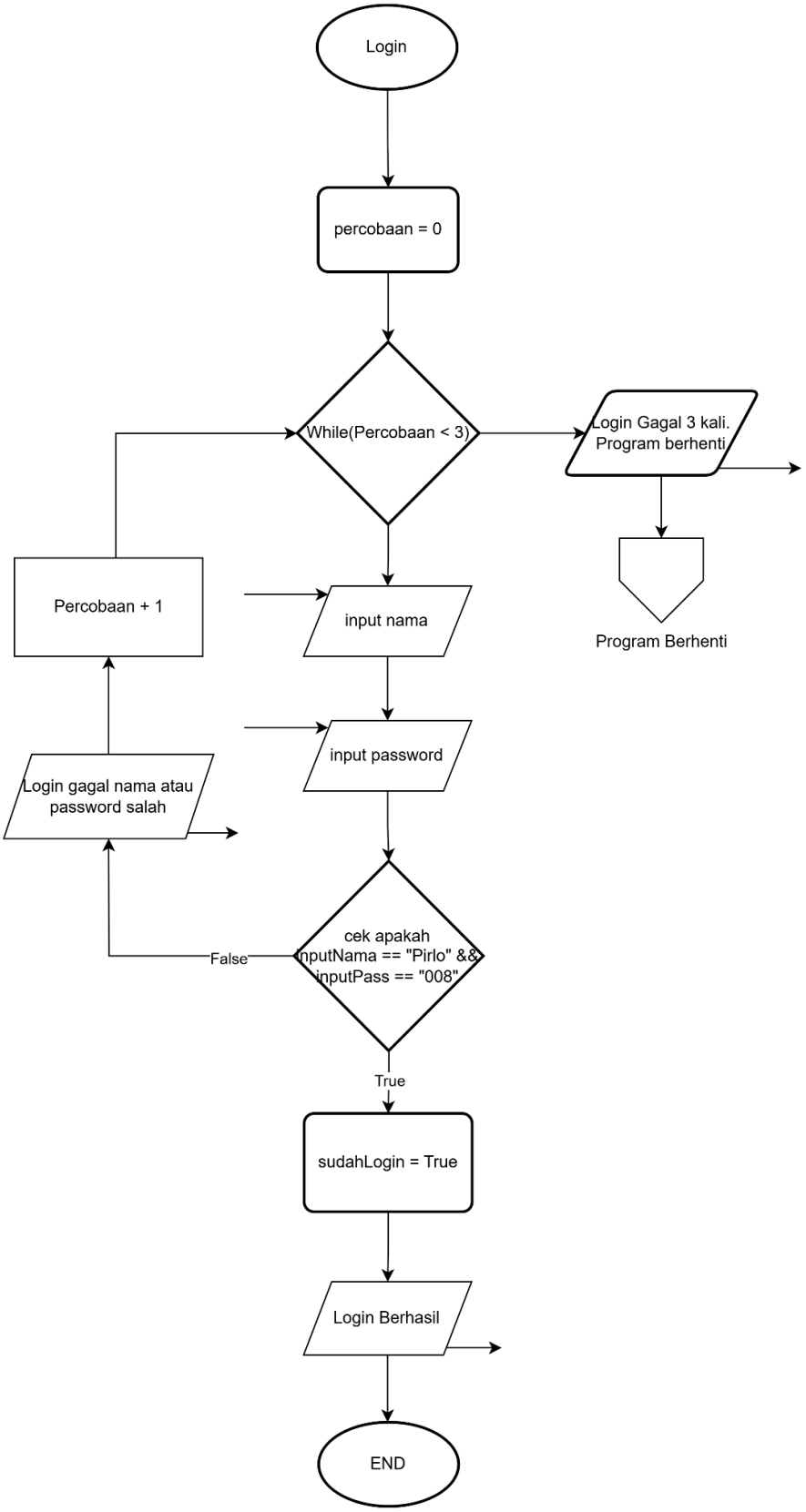
SAMARINDA

2026

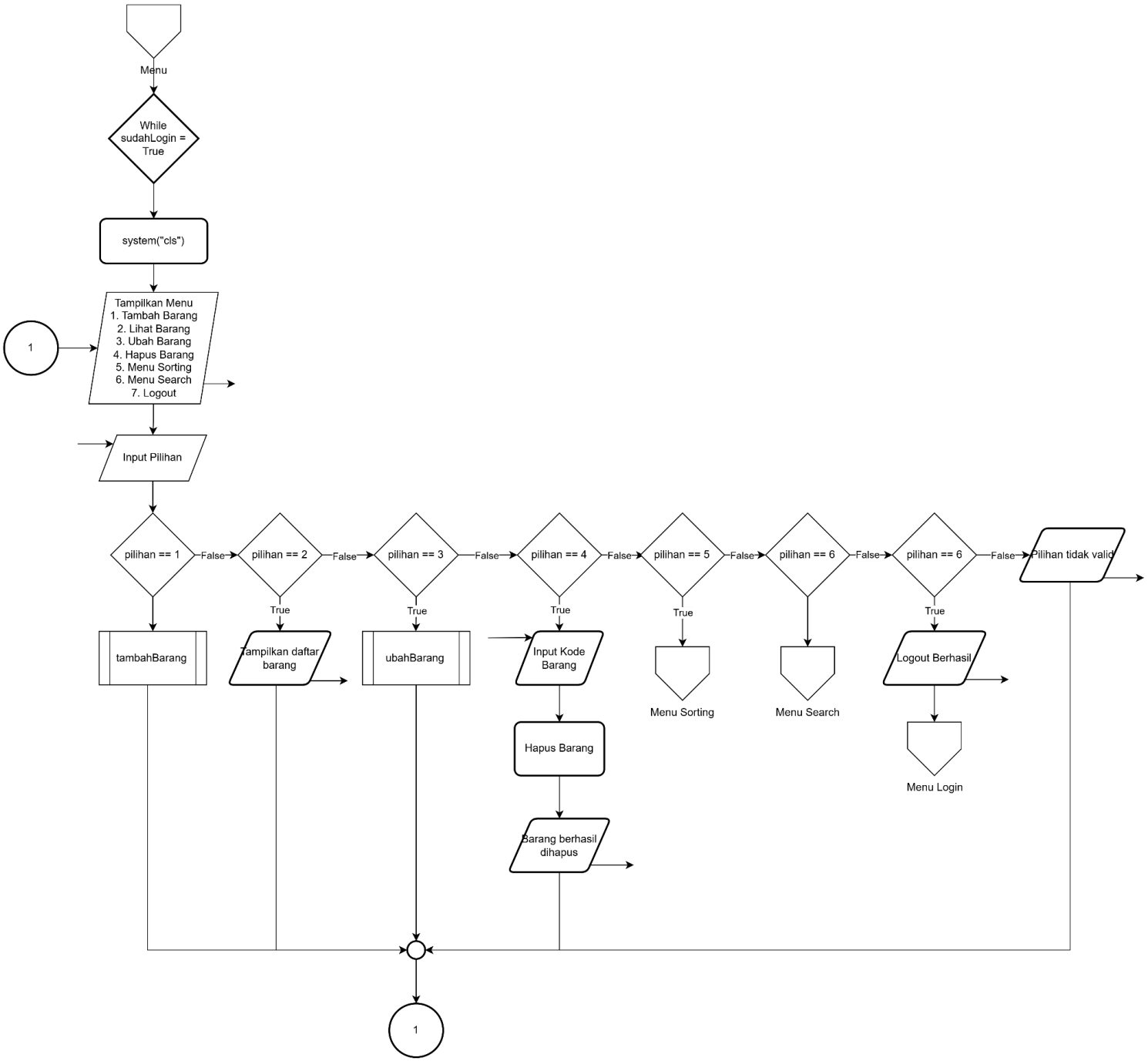
1. Flowchart



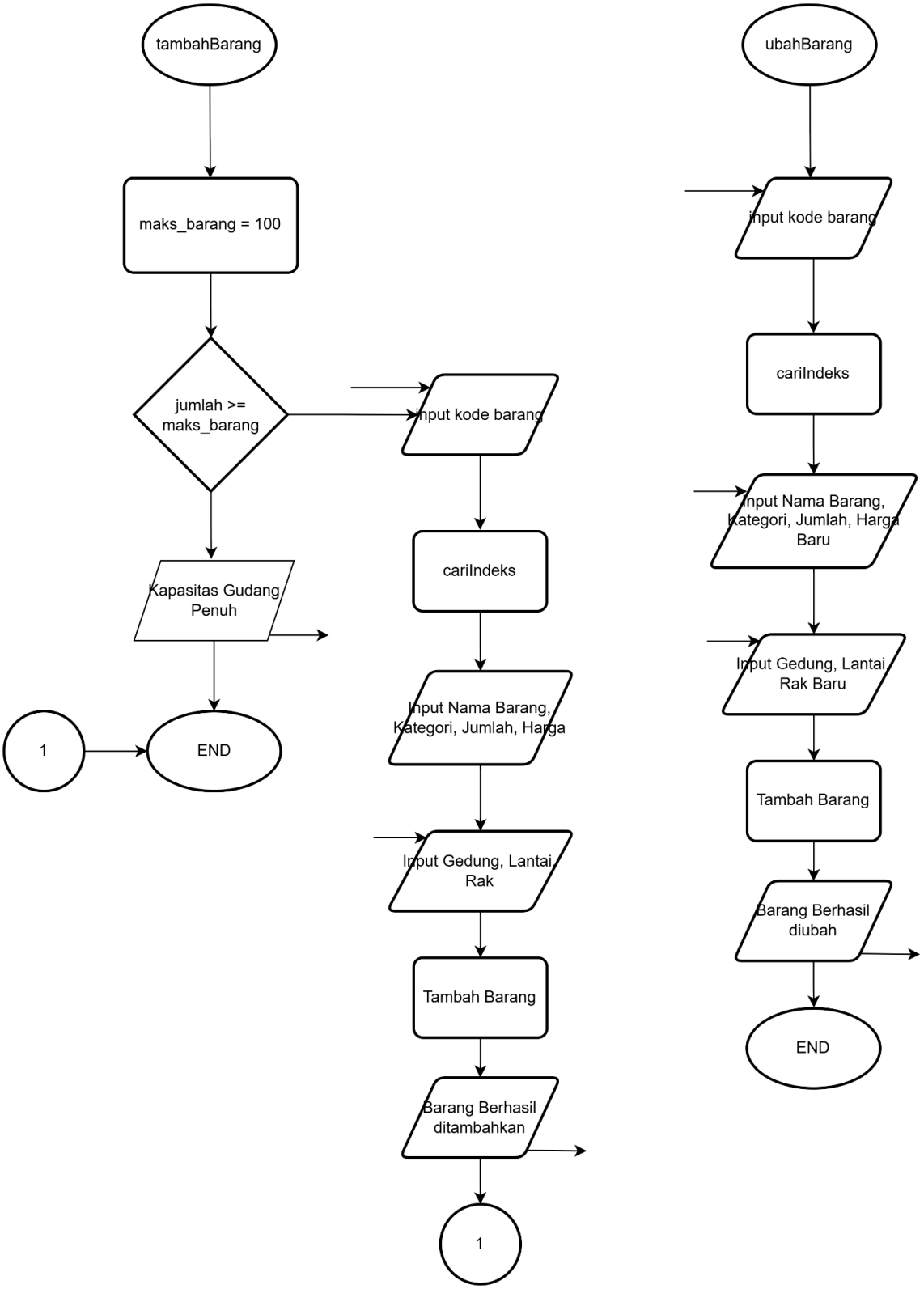
Gambar 1.1 Flowchart



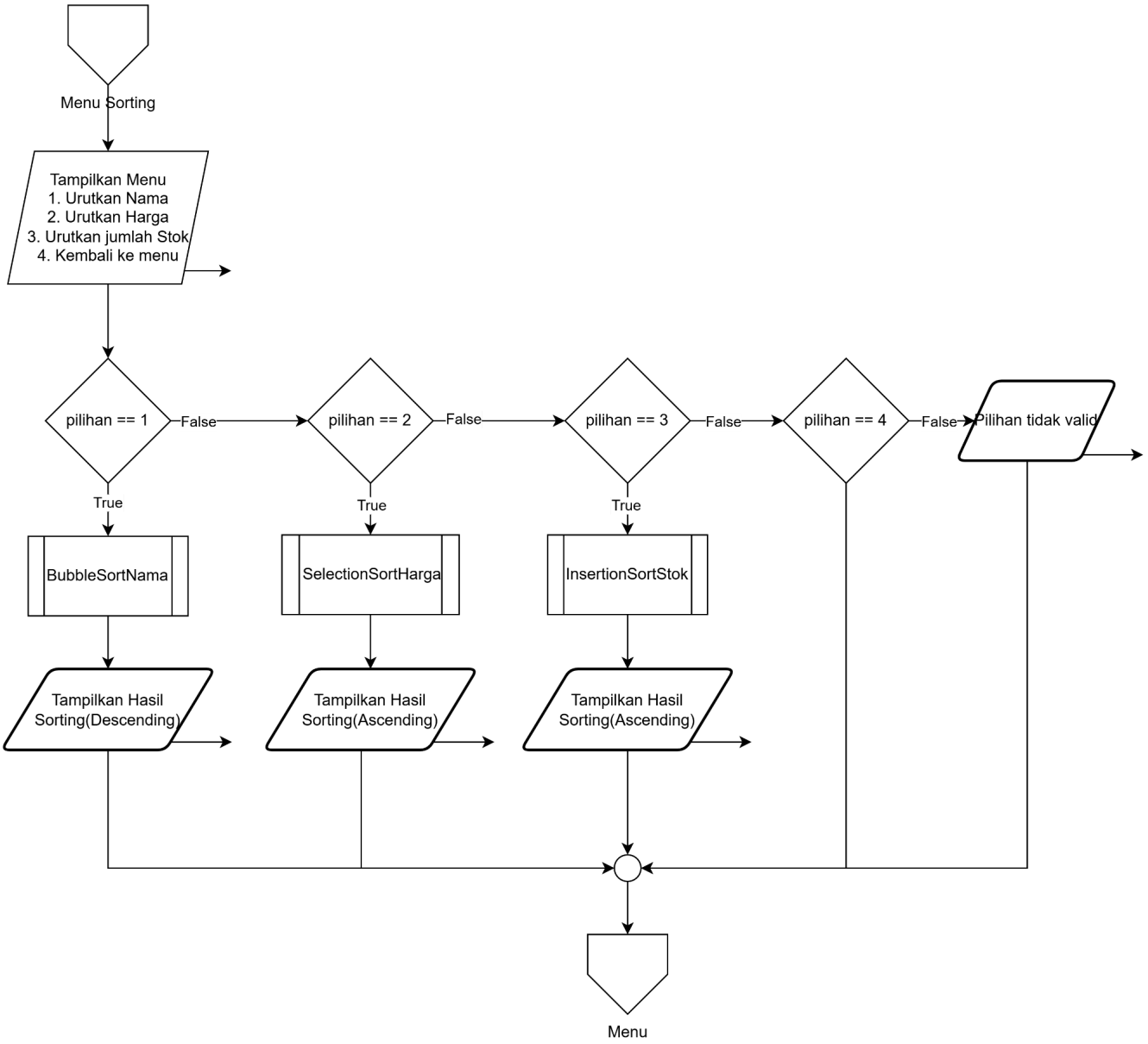
Gambar 1.2 Flowchart



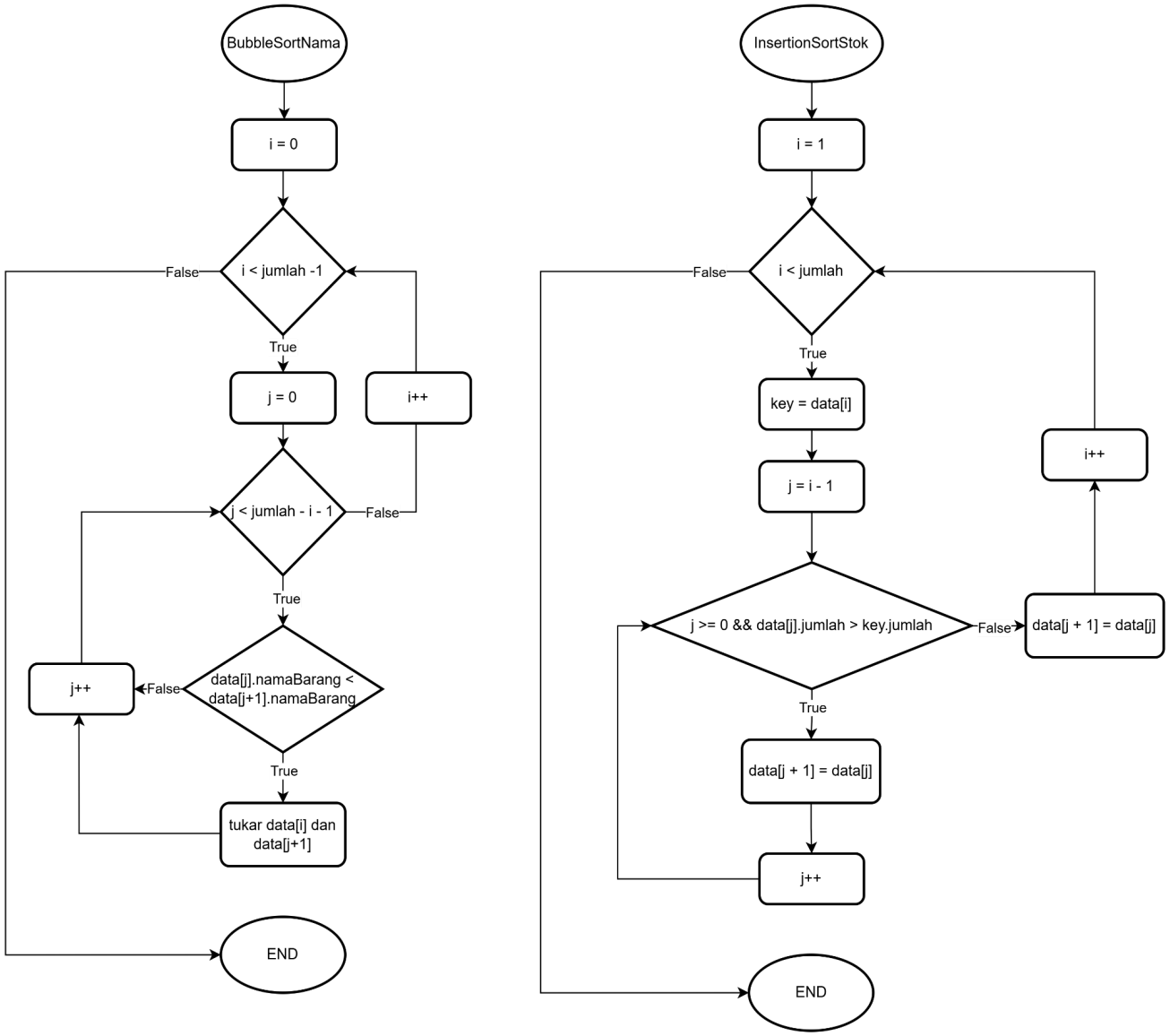
Gambar 1.3 Flowchart



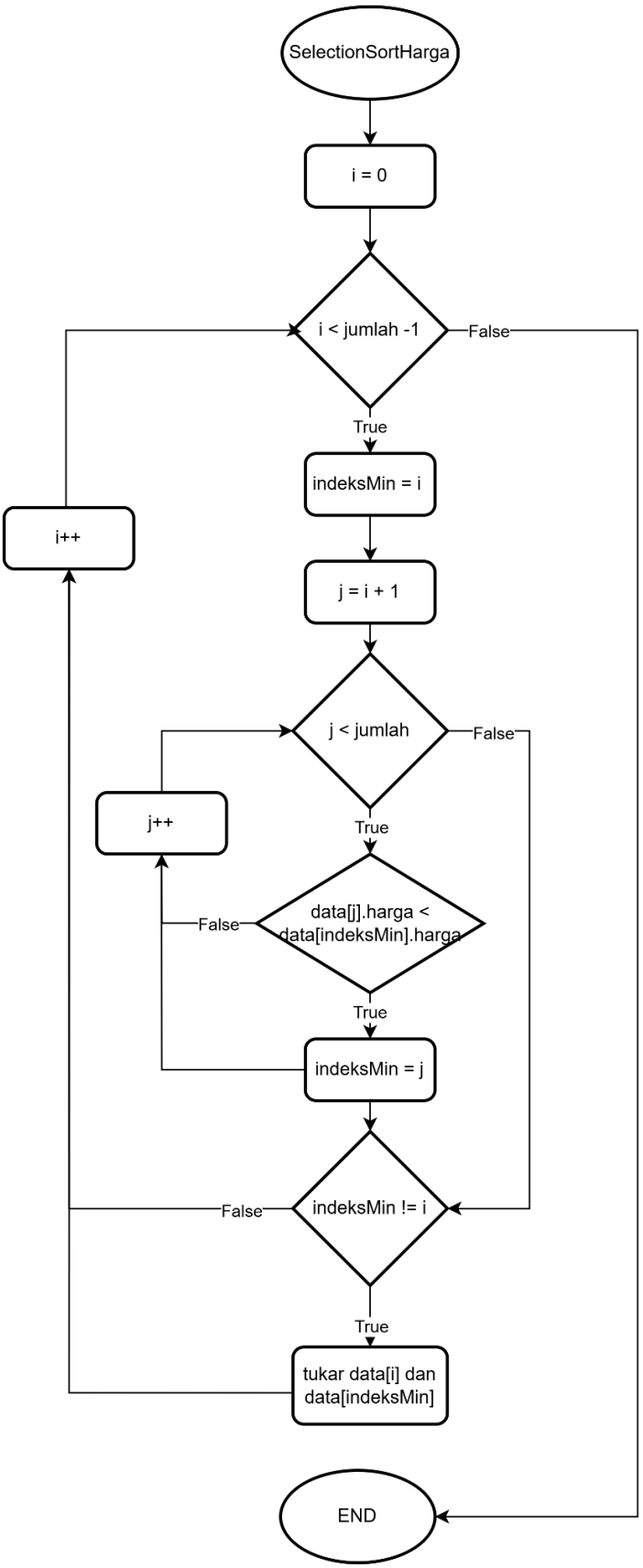
Gambar 1.4 Flowchart



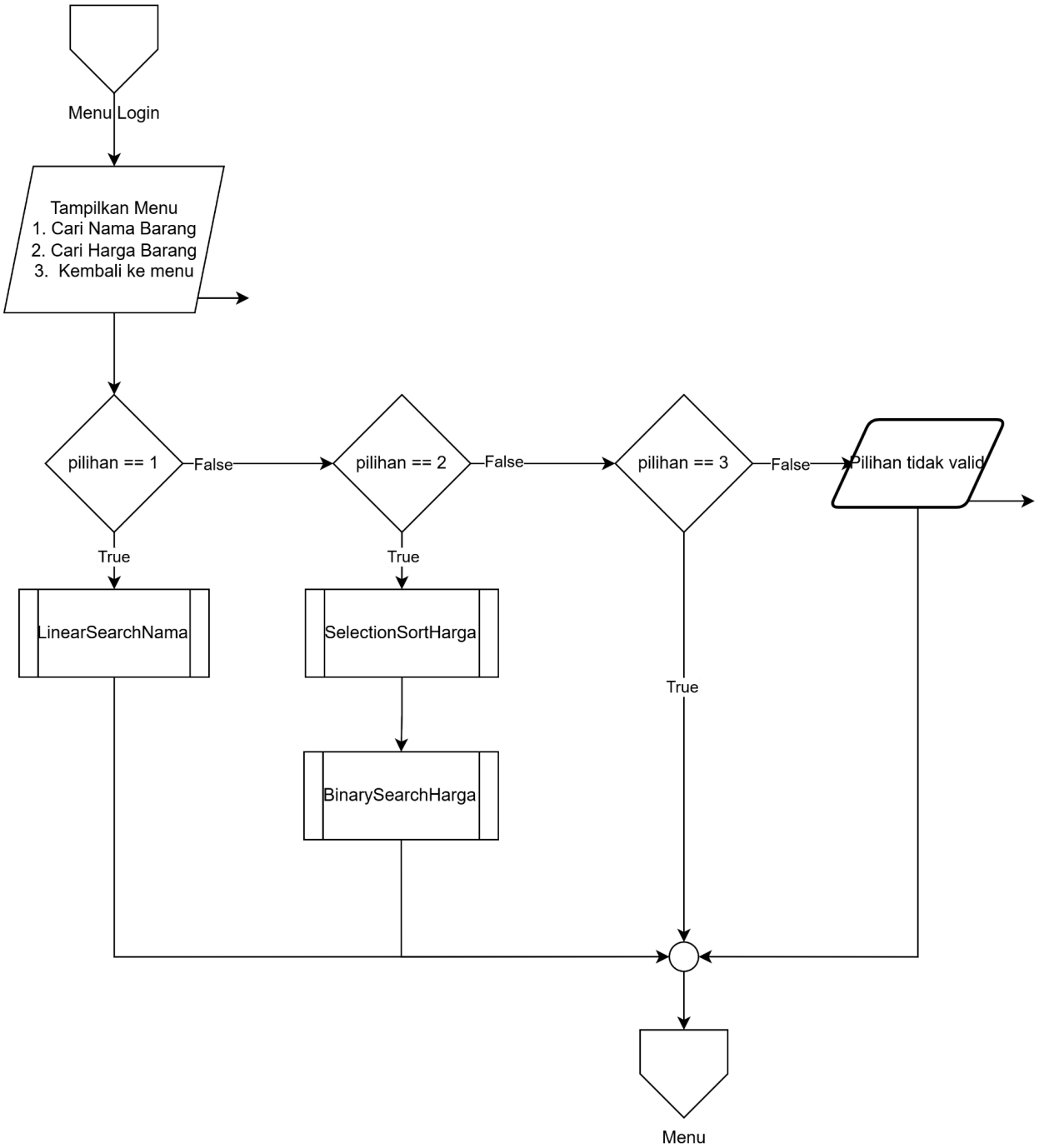
Gambar 1.5 Flowchart



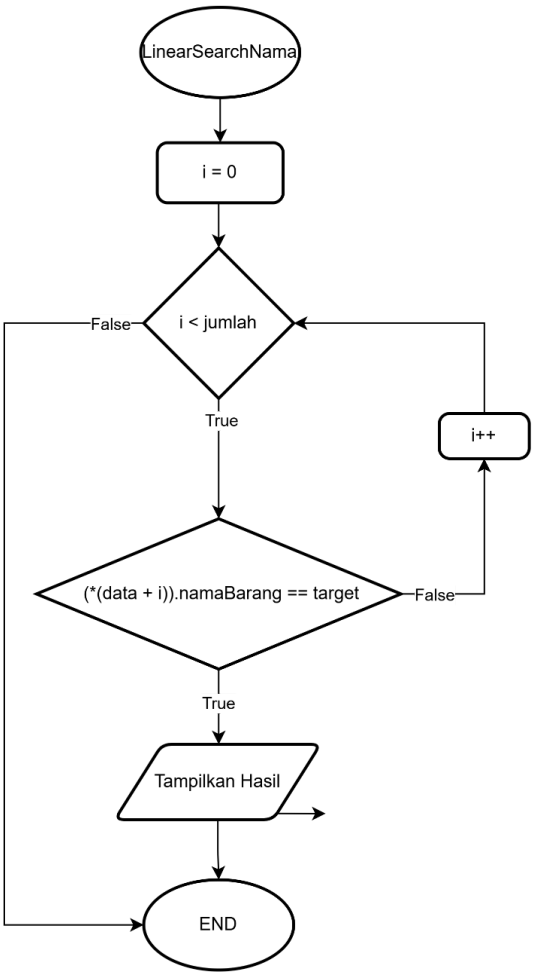
Gambar 1.6 Flowchart

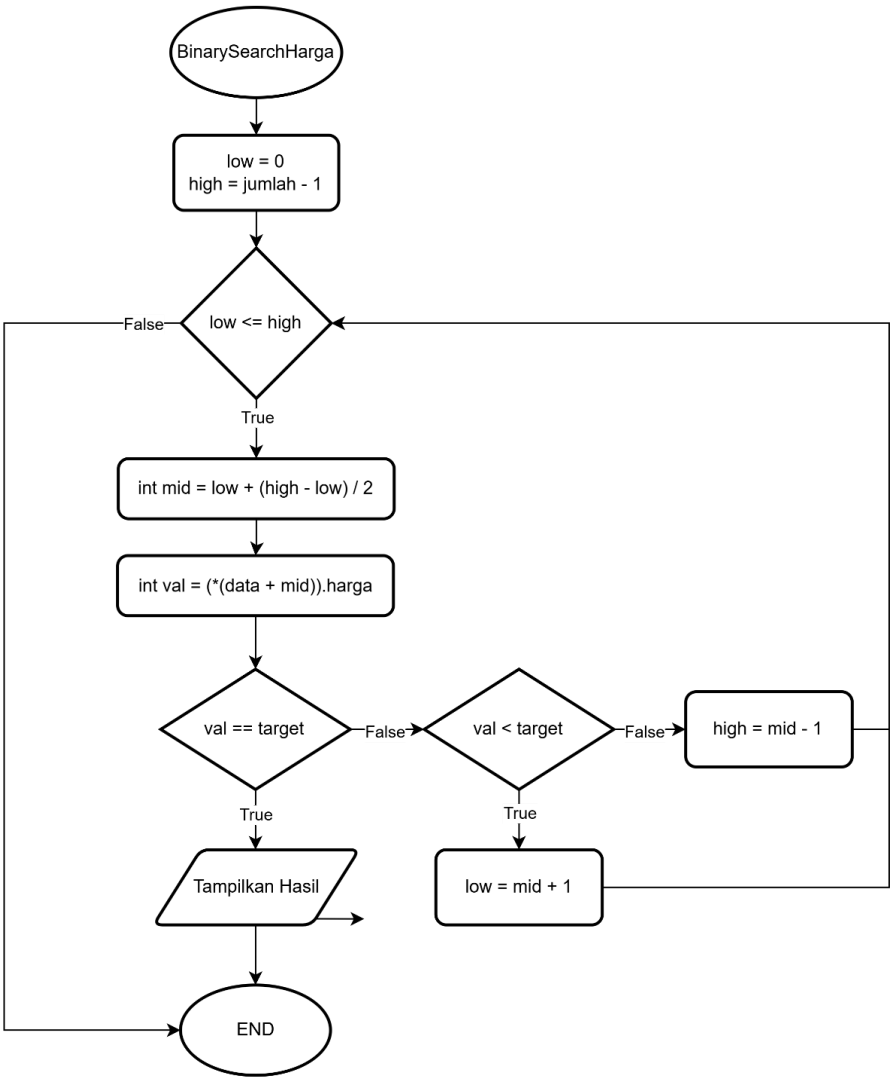


Gambar 1.7 Flowchart



Gambar 1.8 Flowchart





Gambar 1.9 Flowchart

2. Deskripsi Singkat Program

Program ini merupakan aplikasi manajemen inventaris gudang elektronik yang dilengkapi dengan sistem login. Pengguna harus memasukkan username dan password dengan batas maksimal tiga kali percobaan sebelum akses diberikan. Setelah berhasil login, tersedia menu utama yang akan terus berjalan hingga pengguna memilih opsi logout. Program ini menyediakan fitur tambah, lihat, ubah, dan hapus data barang lengkap dengan informasi lokasi penyimpanan.

3. Source Code

A. Fitur Login

Fitur Login menggunakan fungsi rekursif `prosesLogin` dengan parameter `percobaan`. Jika `percobaan >= 3`, fungsi mengembalikan `false` dan program berhenti. Jika belum, program meminta input username dan password; jika sesuai ("Pirlo" dan "008"), login berhasil dan fungsi mengembalikan `true`. Jika salah, fungsi memanggil dirinya sendiri dengan `percobaan + 1` untuk mengulangi proses hingga berhasil atau percobaan habis.

Source Code:

```
bool prosesLogin(int percobaan) {
    try {
        if (percobaan >= 3) {
            throw LoginGagalException(0);
        }

        string inputNama, inputPass;
        cout << "\n--- Login (Percobaan " << percobaan + 1 << "/3) ---";
        inputNama = inputStringValid("\nUsername: ");
        inputPass = inputStringValid("Password: ");

        if (inputNama == "Pirlo" && inputPass == "008") {
            cout << "\nLogin Berhasil! Selamat datang." << endl;
            cout << "Tekan Enter untuk melanjutkan...";
            cin.ignore();
            cin.get();
            return true;
        } else {
            throw LoginGagalException(3 - (percobaan + 1));
        }
```

```
    }
    catch (const LoginGagalException& e) {
        cout << "\n" << e.what() << endl;
        if (percobaan >= 2) {
            cout << "\nGagal login 3 kali. Program berhenti.";
            return false;
        }
        cout << "Tekan Enter untuk mencoba lagi...";
        cin.ignore();
        cin.get();
        return prosesLogin(percobaan + 1);
    }
    catch (const exception& e) {
        cout << "\nTerjadi kesalahan sistem: " << e.what() << endl;
        return false;
    }
}
```

B. Fitur Tampilan Menu

Prosedur `tampilkanMenuUtama` hanya berfungsi menampilkan daftar pilihan menu (1-5). Fungsi `main` bertugas sebagai pengendali utama program: menginisialisasi data, mengatur alur login, dan menampilkan menu utama dalam sebuah loop. Berdasarkan input pengguna, `main` hanya memanggil fungsi-fungsi lain seperti `tambahBarang`, `lihatBarang`, `ubahBarang`, `hapusBarang`, atau menangani proses logout dan keluar program.

Source Code:

```
void tampilkanMenuUtama() {
    cout << "\n1. Tambah Barang";
    cout << "\n2. Lihat Barang";
    cout << "\n3. Ubah Barang";
    cout << "\n4. Hapus Barang";
    cout << "\n5. Sorting Barang";
    cout << "\n6. Cari Barang";
    cout << "\n7. Logout";
}

int main() {
    const int MAKS_BARANG = 100;
    ItemInventaris dataBarang[MAKS_BARANG];
    int jumlahBarang = 0;

    dataBarang[0] = {"EL001", "Laptop Asus", "Komputer", 10, 5000000, {"Gudang
A", "1", "01"}, true};
    dataBarang[1] = {"EL002", "Mouse Logitech", "Aksesoris", 50, 150000,
{"Gudang A", "1", "02"}, true};
    dataBarang[2] = {"EL003", "Monitor LG", "Komputer", 0, 2000000, {"Gudang B",
```

```
"2", "05"}, false};
    jumlahBarang = 3;

    bool programAktif = true;

    while (programAktif) {
        bool sudahLogin = false;
        while (!sudahLogin && programAktif) {
            system("cls");
            cout << "\n==================================================";
            cout << "\n   SISTEM MANAJEMEN INVENTARIS GUDANG ELEKTRONIK";
            cout << "\n==================================================";
            cout << "\n1. Login";
            cout << "\n2. Keluar Program";

            int menuAwal = inputMenuValid("\n\nPilihan: ", 1, 2);

            if (menuAwal == 1) {
                sudahLogin = prosesLogin(0);
            } else {
                cout << "\nTerima kasih telah menggunakan program ini.";
                programAktif = false;
            }
        }

        while (sudahLogin) {
            system("cls");
            cout << "\n==================================================";
            cout << "\n   MENU UTAMA INVENTARIS GUDANG ELEKTRONIK";
            cout << "\n==================================================";
            tampilkanMenuUtama();

            int menuUtama = inputMenuValid("\n\nPilihan: ", 1, 7);

            if (menuUtama == 1) tambahBarang(dataBarang, jumlahBarang,
MAKS_BARANG);
            else if (menuUtama == 2) lihatBarang(dataBarang, jumlahBarang);
            else if (menuUtama == 3) ubahBarang(dataBarang, jumlahBarang);
            else if (menuUtama == 4) hapusBarang(dataBarang, &jumlahBarang);
            else if (menuUtama == 5) menuSorting(dataBarang, jumlahBarang);
            else if (menuUtama == 6) menuPencarian(dataBarang, jumlahBarang);
            else if (menuUtama == 7) {
                cout << "\nLogout berhasil.";
                cout << "\nTekan Enter untuk keluar...";
                cin.ignore(); cin.get();
                sudahLogin = false;
            }
        }
    }
    return 0;
}
```

C. Fitur Tambah Barang

Prosedur `tambahBarang` berfungsi menambahkan data inventaris baru ke dalam array. Pertama, program mengecek apakah gudang sudah penuh (`jumlah >= maks`); jika penuh, proses dibatalkan. Jika masih ada ruang, program meminta input `kodeBarang` dan memvalidasinya menggunakan fungsi `cariIndex` agar tidak ada kode duplikat. Selanjutnya, program menginput detail barang (nama, kategori, jumlah, harga) serta lokasi penyimpanan (gedung, lantai, rak), lalu mengatur status `tersedia` berdasarkan nilai jumlah. Terakhir, variabel `jumlah` ditambah 1 dan pesan sukses ditampilkan.

Source Code:

```
void tambahBarang(ItemInventaris data[], int &jumlah, int maks) {
    try {
        cout << "\n==================================================";
        cout << "\n   TAMBAH BARANG";
        cout << "\n==================================================";

        if (jumlah >= maks) throw KapasitasPenuhException();

        data[jumlah].kodeBarang = inputStringValid("\nKode Barang: ");

        if (cariIndex(data, 0, jumlah, data[jumlah].kodeBarang) != -1) throw
KodeDuplikatException();

        data[jumlah].namaBarang = inputStringValid("Nama Barang: ");
        data[jumlah].kategori = inputStringValid("Kategori: ");
        data[jumlah].jumlah = inputIntValid("Jumlah: ");
        data[jumlah].harga = inputIntValid("Harga: ");

        cout << "\n--- Lokasi Penyimpanan ---";
        data[jumlah].alamat.gedung = inputStringValid("\nGedung: ");
        data[jumlah].alamat.lantai = inputStringValid("Lantai: ");
        data[jumlah].alamat.rak = inputStringValid("Rak: ");

        data[jumlah].tersedia = (data[jumlah].jumlah > 0);
        jumlah++;

        cout << "\nBarang berhasil ditambahkan!";
    }
    catch (const InventarisException& e) { cout << "\n" << e.what(); }
    catch (const exception& e) { cout << "\nTerjadi kesalahan: " << e.what(); }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.ignore(); cin.get();
}
```

D. Fitur Lihat Barang

Prosedur `lihatBarang` menampilkan daftar inventaris gudang dalam format tabel. Jika tidak ada data (`jumlah == 0`), program menampilkan pesan "Belum ada data barang". Jika ada data, program membuat tabel, kemudian mengisi setiap baris dengan informasi barang (kode, nama, kategori, jumlah, harga, lokasi gedung-lantai-rak, serta status "Tersedia" atau "Habis" berdasarkan nilai `tersedia`).

Source Code:

```
void lihatBarang(ItemInventaris data[], int jumlah) {
    cout << "\n==================================================";
    cout << "\n   DAFTAR INVENTARIS GUDANG";
    cout << "\n==================================================\n";
    if (jumlah == 0) { cout << "\nBelum ada data barang."; }
    else {
        Table table;
        table.add_row({"Kode", "Nama Barang", "Kategori", "Jumlah", "Harga",
"Lokasi", "Status"});
        for (size_t i = 0; i < table[0].size(); ++i)
table[0][i].format().font_align(FontAlign::center).font_style({FontStyle::bold})
;
        for (int i = 0; i < jumlah; i++) {
            string lokasi = data[i].alamat.gedung + "-" + data[i].alamat.lantai
+ "-" + data[i].alamat.rak;
            string status = data[i].tersedia ? "Tersedia" : "Habis";
            table.add_row({data[i].kodeBarang, data[i].namaBarang,
data[i].kategori, to_string(data[i].jumlah), "Rp " + to_string(data[i].harga),
lokasi, status});
        }
        cout << table << endl;
    }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.ignore(); cin.get();
}
```

E. Fitur Ubah Data Barang

Prosedur `ubahBarang` berfungsi memperbarui data barang yang sudah ada. Program pertama-tama mengecek apakah data kosong (`jumlah == 0`), lalu meminta input `kodeCari` untuk mencari barang menggunakan fungsi `cariIndex`. Jika barang ditemukan (`posisiBarang != -1`), program mengizinkan pengguna menginput ulang seluruh detail barang (nama, kategori, jumlah, harga, dan lokasi penyimpanan), kemudian memperbarui status `tersedia` berdasarkan nilai jumlah. Jika barang tidak ditemukan, program menampilkan pesan kesalahan. Terakhir, program menampilkan konfirmasi hasil operasi dan menunggu pengguna menekan Enter untuk melanjutkan.

Source Code:

```
void ubahBarang(ItemInventaris data[], int jumlah) {
    try {
        cout << "\n==================================================";
        cout << "\n   UBAH BARANG";
        cout << "\n==================================================";
        if (jumlah == 0) throw BarangTidakDitemukanException();

        string kodeCari = inputStringValid("\nMasukkan kode barang: ");
        int posisiBarang = cariIndex(data, 0, jumlah, kodeCari);
        if (posisiBarang == -1) throw BarangTidakDitemukanException();

        data[posisiBarang].namaBarang = inputStringValid("\nNama Barang baru:
");
        data[posisiBarang].kategori = inputStringValid("Kategori baru: ");
        data[posisiBarang].jumlah = inputIntValid("Jumlah baru: ");
        data[posisiBarang].harga = inputIntValid("Harga baru: ");

        data[posisiBarang].alamat.gedung = inputStringValid("Gedung baru: ");
        data[posisiBarang].alamat.lantai = inputStringValid("Lantai baru: ");
        data[posisiBarang].alamat.rak = inputStringValid("Rak baru: ");

        data[posisiBarang].tersedia = (data[posisiBarang].jumlah > 0);
        cout << "\nData berhasil diubah.";
    }
    catch (const InventarisException& e) { cout << "\n" << e.what(); }
    catch (const exception& e) { cout << "\nTerjadi kesalahan: " << e.what(); }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.ignore(); cin.get();
}
```

F. Fitur Hapus Barang

Prosedur `hapusBarang` berfungsi menghapus data barang berdasarkan kode. Program pertama-tama mengecek apakah data kosong (`jumlah == 0`), lalu meminta input `kodeHapus` dan mencarinya menggunakan fungsi `cariIndex`. Jika barang ditemukan, program meminta konfirmasi pengguna (y/n); jika dikonfirmasi, data dihapus dengan menggeser elemen array setelah posisi tersebut ke kiri (`data[i] = data[i + 1]`), variabel `jumlah` dikurangi 1, dan pesan sukses ditampilkan. Jika barang tidak ditemukan atau pengguna membatalkan, program menampilkan pesan sesuai kondisi. Terakhir, program menunggu pengguna menekan Enter untuk melanjutkan.

Source Code:

```
void hapusBarang(ItemInventaris* data, int* jumlah) {
    try {
        cout << "\n==================================================";
        cout << "\n   HAPUS BARANG";
        cout << "\n==================================================";
        if (*jumlah == 0) throw BarangTidakDitemukanException();

        string kodeHapus = inputStringValid("\nMasukkan kode barang: ");
        int posisiBarang = cariIndex(data, 0, *jumlah, kodeHapus);
        if (posisiBarang == -1) throw BarangTidakDitemukanException();

        string konfirmasi = inputStringValid("Yakin ingin menghapus? (y/n): ");
        if (konfirmasi == "y" || konfirmasi == "Y") {
            // Menggunakan pointer arithmetic untuk shifting array
            for (int i = posisiBarang; i < *jumlah - 1; i++) {
                data[i] = data[i + 1];
            }
            (*jumlah)--;
            cout << "\nBarang berhasil dihapus!";
        } else { cout << "\nPenghapusan dibatalkan."; }
    }
    catch (const InventarisException& e) { cout << "\n" << e.what(); }
    catch (const exception& e) { cout << "\nTerjadi kesalahan: " << e.what(); }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.ignore(); cin.get();
}
```

G. Fitur Menu Sorting

Prosedur `tampilkanDataTerurut` berfungsi menampilkan data inventaris dalam format tabel yang rapi dengan header yang diformat tebal dan rata tengah. Jika data kosong, program menampilkan pesan informasi; jika ada, program mengolah data (menggabungkan alamat, mengonversi status boolean, memformat harga) lalu menampilkannya baris per baris, lalu `menuSorting` berfungsi menyediakan menu untuk mengurutkan data inventaris berdasarkan Nama (Z ke A), Harga (kecil ke besar), atau Stok (sedikit ke banyak). Program membuat salinan data agar data asli tidak berubah, memanggil fungsi sorting yang sesuai dengan pilihan pengguna, lalu menampilkan hasilnya melalui `tampilkanDataTerurut`.

Source Code:

```
void tampilkanDataTerurut(ItemInventaris data[], int jumlah, string judul) { //
Prosedur
    cout << "\n==================================================";
    cout << "\n    " << judul;
    cout << "\n==================================================";
    cout << "\n";

    if (jumlah == 0) {
        cout << "\nBelum ada data barang.";
    } else {
        Table table;

        table.add_row({"Kode", "Nama Barang", "Kategori", "Jumlah", "Harga",
"Lokasi", "Status"});

        for (size_t i = 0; i < table[0].size(); ++i) {
            table[0][i].format()
                .font_align(FontAlign::center)
                .font_style({FontStyle::bold});
        }

        for (int i = 0; i < jumlah; i++) {
            string lokasi = data[i].alamat.gedung + "-" +
                            data[i].alamat.lantai + "-" +
                            data[i].alamat.rak;
            string status = data[i].tersedia ? "Tersedia" : "Habis";

            table.add_row({
                data[i].kodeBarang,
                data[i].namaBarang,
                data[i].kategori,
                to_string(data[i].jumlah),
                "Rp " + to_string(data[i].harga),
```

```
                lokasi,
                status
            });
        }

        cout << table << endl;
    }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.get();
}

void menuSorting(ItemInventaris data[], int jumlah) { // Prosedur
    cout << "\n==================================================";
    cout << "\n   MENU SORTING INVENTARIS";
    cout << "\n==================================================";

    if (jumlah == 0) {
        cout << "\nBelum ada data barang untuk diurutkan.";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
        return;
    }

    cout << "\n1. Urutkan Nama Barang (Z ke A)";
    cout << "\n2. Urutkan Harga (Terkecil ke Terbesar)";
    cout << "\n3. Urutkan Jumlah Stok (Sedikit ke Banyak)";
    cout << "\n4. Kembali ke Menu Utama";
    cout << "\n\nPilihan: ";

    int pilihan;
    cin >> pilihan;
    cin.ignore();

    ItemInventaris dataSalinan[100];
    for (int i = 0; i < jumlah; i++) {
        dataSalinan[i] = data[i];
    }

    if (pilihan == 1) {
        bubbleSortNamaDescending(dataSalinan, jumlah);
        tampilkanDataTerurut(dataSalinan, jumlah, "HASIL SORTING: NAMA BARANG
(DESCENDING)");
    } else if (pilihan == 2) {
        selectionSortHargaAscending(dataSalinan, jumlah);
        tampilkanDataTerurut(dataSalinan, jumlah, "HASIL SORTING: HARGA
(ASCENDING)");
    } else if (pilihan == 3) {
        insertionSortJumlahAscending(dataSalinan, jumlah);
        tampilkanDataTerurut(dataSalinan, jumlah, "HASIL SORTING: JUMLAH STOK
(ASCENDING)");
    } else if (pilihan == 4) {
```

```
        return;
    } else {
```

H. Fitur Sorting Nama(Descending)

Fungsi `bubbleSortNamaDescending` mengurutkan data inventaris berdasarkan `namaBarang` secara descending (Z ke A) menggunakan algoritma Bubble Sort. Program melakukan perulangan bersarang untuk membandingkan nama barang yang bersebelahan; jika nama sebelumnya lebih kecil secara alfabetis (`<`), maka kedua data ditukar. Proses ini diulang hingga tidak ada lagi pertukaran yang terjadi (`swapped = false`), yang menandakan data sudah terurut, atau hingga seluruh elemen selesai diproses.

Source Code:

```
void bubbleSortNamaDescending(ItemInventaris data[], int jumlah) { // Fungsi
Rekursif Iteratif
    if (jumlah <= 1) return;

    bool swapped;
    for (int i = 0; i < jumlah - 1; i++) {
        swapped = false;
        for (int j = 0; j < jumlah - i - 1; j++) {
            if (data[j].namaBarang < data[j + 1].namaBarang) {
                ItemInventaris temp = data[j];
                data[j] = data[j + 1];
                data[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

I. Fitur Sorting Harga(Ascending)

Fungsi `selectionSortHargaAscending` mengurutkan data inventaris berdasarkan `harga` secara ascending (terkecil ke terbesar) menggunakan algoritma Selection Sort. Program mencari elemen dengan harga minimum pada bagian array yang belum terurut, lalu menukarnya dengan elemen di posisi saat ini. Proses ini diulang untuk setiap posisi hingga seluruh data terurut, dengan penukaran hanya dilakukan jika diperlukan (`indeksMin != i`).

Source Code:

```
void selectionSortHargaAscending(ItemInventaris data[], int jumlah) { //
Prosedur
    if (jumlah <= 1) return;

    for (int i = 0; i < jumlah - 1; i++) {
        int indeksMin = i;
        for (int j = i + 1; j < jumlah; j++) {
            if (data[j].harga < data[indeksMin].harga) {
                indeksMin = j;
            }
        }
        if (indeksMin != i) {
            ItemInventaris temp = data[i];
            data[i] = data[indeksMin];
            data[indeksMin] = temp;
        }
    }
}
```

J. Fitur Sorting Jumlah Stok(Ascending)

Fungsi `insertionSortJumlahAscending` mengurutkan data inventaris berdasarkan `jumlah` stok secara ascending (sedikit ke banyak) menggunakan algoritma Insertion Sort. Program mengambil elemen satu per satu mulai dari indeks ke-1 sebagai `key`, lalu membandingkannya dengan elemen-elemen sebelumnya yang sudah terurut. Jika elemen sebelumnya memiliki nilai `jumlah` lebih besar, elemen tersebut digeser ke kanan untuk memberi ruang bagi `key`. Proses ini berlanjut hingga posisi yang tepat ditemukan, lalu `key` ditempatkan di posisi tersebut.

Source Code:

```
void insertionSortJumlahAscending(ItemInventaris data[], int jumlah) { //
Prosedur
    if (jumlah <= 1) return;

    for (int i = 1; i < jumlah; i++) {
        ItemInventaris key = data[i];
        int j = i - 1;
        while (j >= 0 && data[j].jumlah > key.jumlah) {
            data[j + 1] = data[j];
            j = j - 1;
        }
        data[j + 1] = key;
    }
}
```

K. Fitur Menu Searching

Prosedur `menuPencarian` menyediakan fitur pencarian data inventaris berdasarkan 'Nama Barang' atau 'Harga' dalam menu interaktif. Untuk pencarian nama, program menggunakan `linearSearchNama` untuk menelusuri data secara langsung; jika ditemukan, data ditampilkan dalam format tabel. Untuk pencarian harga, program terlebih dahulu membuat salinan data dan mengurutkannya dengan `selectionSortHargaAscending`, lalu menerapkan `binarySearchHarga` agar pencarian lebih efisien. Program menggunakan perulangan `do-while` agar menu terus muncul hingga pengguna memilih opsi kembali, dan setiap hasil pencarian diakhiri dengan menunggu input Enter untuk melanjutkan.

Source Code:

```
void menuPencarian(ItemInventaris data[], int jumlah) {
    int pilihan;
    do {
        system("cls");
        cout << "\n==================================================";
        cout << "\n   MENU PENCARIAN (SEARCHING)";
        cout << "\n==================================================";
        cout << "\n1. Cari berdasarkan Nama Barang";
        cout << "\n2. Cari berdasarkan Harga";
        cout << "\n3. Kembali ke Menu Utama";

        pilihan = inputMenuValid("\n\nPilihan: ", 1, 3);
```

```
        try {
            if (pilihan == 1) {
                string namaCari = inputStringValid("\nMasukkan Nama Barang: ");
                int idx = linearSearchNama(data, jumlah, namaCari);
                if (idx != -1) {
                    cout << "\nBarang ditemukan!" << endl;
                    Table table; table.add_row({"Kode", "Nama Barang",
"Kategori", "Jumlah", "Harga", "Lokasi", "Status"});
                    string lokasi = data[idx].alamat.gedung + "-" +
data[idx].alamat.lantai + "-" + data[idx].alamat.rak;
                    table.add_row({data[idx].kodeBarang, data[idx].namaBarang,
data[idx].kategori, to_string(data[idx].jumlah), "Rp " +
to_string(data[idx].harga), lokasi, data[idx].tersedia ? "Tersedia" : "Habis"});
                    cout << table << endl;
                } else throw BarangTidakDitemukanException();
            }
            else if (pilihan == 2) {
                int hargaCari = inputIntValid("\nMasukkan Harga Barang (Rp): ");
                ItemInventaris dataTemp[100];
                for (int i = 0; i < jumlah; i++) dataTemp[i] = data[i];
                selectionSortHargaAscending(dataTemp, jumlah);
                int idx = binarySearchHarga(dataTemp, jumlah, hargaCari);
                if (idx != -1) {
                    cout << "\nBarang ditemukan dengan harga Rp " <<
dataTemp[idx].harga << "!" << endl;
                    Table table; table.add_row({"Kode", "Nama Barang",
"Kategori", "Jumlah", "Harga", "Lokasi", "Status"});
                    string lokasi = dataTemp[idx].alamat.gedung + "-" +
dataTemp[idx].alamat.lantai + "-" + dataTemp[idx].alamat.rak;
                    table.add_row({dataTemp[idx].kodeBarang,
dataTemp[idx].namaBarang, dataTemp[idx].kategori,
to_string(dataTemp[idx].jumlah), "Rp " + to_string(dataTemp[idx].harga), lokasi,
dataTemp[idx].tersedia ? "Tersedia" : "Habis"});
                    cout << table << endl;
                } else throw BarangTidakDitemukanException();
            }
            else if (pilihan == 3) return;
            else throw InputInvalidException("Pilihan menu");

            cout << "\nTekan Enter untuk melanjutkan...";
            cin.ignore(); cin.get();
        }
        catch (const exception& e) {
            cout << "\n" << e.what();
            cout << "\nTekan Enter untuk melanjutkan...";
            cin.ignore(); cin.get();
        }
    } while (pilihan != 3);
}
```

L. Fitur Searching Nama

Fungsi `linearSearchNama` mencari data inventaris berdasarkan `namaBarang` menggunakan algoritma Linear Search dengan pointer arithmetic. Program membandingkan setiap elemen secara berurutan; jika ditemukan kecocokan dengan `target`, fungsi mengembalikan indeksnya, sedangkan jika tidak ditemukan setelah seluruh data diperiksa, fungsi mengembalikan `-1`.

Source Code:

```
int linearSearchNama(ItemInventaris* data, int jumlah, string target) { // Linear Search untuk kata/huruf (namaBarang)
    for (int i = 0; i < jumlah; i++) {
        if ((*(data + i)).namaBarang == target) {
            return i;
        }
    }
    return -1;
}
```

M. Fitur Searching Harga

Fungsi `binarySearchHarga` mencari data inventaris berdasarkan `harga` menggunakan algoritma Binary Search dengan pointer arithmetic. Program bekerja pada data yang sudah terurut dengan mempersempit rentang pencarian (`low` dan `high`) dan membandingkan nilai tengah (`mid`) dengan `target`; jika nilai lebih kecil, pencarian dilanjutkan ke kanan, jika lebih besar ke kiri. Jika harga ditemukan, fungsi mengembalikan indeksnya; jika tidak ditemukan setelah rentang habis, fungsi mengembalikan `-1`.

Source Code:

```
int binarySearchHarga(ItemInventaris* data, int jumlah, int target) { // Binary Search untuk angka (harga)
    int low = 0, high = jumlah - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int val = (*(data + mid)).harga;
        if (val == target) {
            return mid;
        } else if (val < target) {
```

```
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}
```

N. Fitur Error Handling

File `ErrorHandling.h` mendefinisikan sistem penanganan error dan validasi input terpusat. Program membuat kelas exception kustom (`InventarisException` dan turunannya) untuk menangani error spesifik seperti login gagal, kode duplikat, atau input tidak valid. File ini juga menyediakan fungsi validasi input (`inputStringValid`, `inputIntValid`, `inputMenuValid`) yang memastikan data yang dimasukkan pengguna sudah sesuai sebelum diproses lebih lanjut.

```
#ifndef ERRORHANDLING_H
#define ERRORHANDLING_H

#include <exception>
#include <string>
#include <iostream>
#include <limits>

class InventarisException : public std::exception {
protected: std::string pesan;
public:
    InventarisException(const std::string& msg) : pesan(msg) {}
    const char* what() const noexcept override { return pesan.c_str(); }
    virtual ~InventarisException() noexcept = default;
};

class LoginGagalException : public InventarisException {
public: LoginGagalException(int sisa) : InventarisException("Gagal login! Sisa
percobaan: " + std::to_string(sisa)) {} };

class KapasitasPenuhException : public InventarisException {
public: KapasitasPenuhException() : InventarisException("Kapasitas gudang
penuh!") {} };

class KodeDuplikatException : public InventarisException {
public: KodeDuplikatException() : InventarisException("Kode barang sudah
digunakan!") {} };
```

```
class BarangTidakDitemukanException : public InventarisException {
public: BarangTidakDitemukanException() : InventarisException("Barang tidak
ditemukan!") {} };

class InputInvalidException : public InventarisException {
public: InputInvalidException(const std::string& field) :
InventarisException("Input tidak valid untuk " + field + "!") {} };

// Validasi string tidak boleh kosong
inline std::string inputStringValid(const std::string& prompt) {
    std::string val;
    while (true) {
        std::cout << prompt;
        std::getline(std::cin, val);
        if (!val.empty()) return val;
        std::cout << "Input tidak boleh kosong! Coba lagi." << std::endl;
    }
}

// Validasi input angka
inline int inputIntValid(const std::string& prompt) {
    int val;
    while (true) {
        std::cout << prompt;
        if (std::cin >> val) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return val;
        }
        std::cout << "Input harus berupa angka! Coba lagi." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

// Validasi pilihan menu (range min-max)
inline int inputMenuValid(const std::string& prompt, int min, int max) {
    std::string line;
    while (true) {
        std::cout << prompt;
        std::getline(std::cin, line);
        if (line.empty()) {
            std::cout << "Input tidak boleh kosong! Masukkan " << min << "-" <<
max << "." << std::endl;
            continue;
        }
        try {
            size_t pos;
            int val = std::stoi(line, &pos);
            if (pos != line.length()) {
                std::cout << "Hanya masukkan angka! Coba lagi." << std::endl;
```

```
                continue;
            }
            if (val >= min && val <= max) return val;
            std::cout << "Pilihan tidak tersedia! Masukkan " << min << "-" <<
max << "." << std::endl;
        } catch (...) {
            std::cout << "Input harus berupa angka! Coba lagi." << std::endl;
        }
    }
}

#endif
```

4. Hasil Output

```
==================================================
   SISTEM MANAJEMEN INVENTARIS GUDANG ELEKTRONIK
==================================================
1. Login
2. Keluar Program

Pilihan: 1

--- Login (Percobaan 1/3) ---
Username: Pirlol
Password: 008
Username atau Password salah!
Tekan Enter untuk mencoba lagi...

--- Login (Percobaan 2/3) ---
Username: Acikiwir
Password: 008
Username atau Password salah!
Tekan Enter untuk mencoba lagi...

--- Login (Percobaan 3/3) ---
Username: Pirlo
Password: hengker tyz
Username atau Password salah!

Gagal login 3 kali. Program berhenti.
Tekan Enter untuk keluar...
```

Gambar 4.1 Output Login Gagal

```
--- Login (Percobaan 1/3) ---
Username: Pirlo
Password: 008

Login Berhasil!
Selamat datang.

Tekan Enter untuk melanjutkan...
```

Gambar 4.2 Output Login Berhasil

```
==================================================
   MENU UTAMA INVENTARIS GUDANG ELEKTRONIK
==================================================
1. Tambah Barang
2. Lihat Barang
3. Ubah Barang
4. Hapus Barang
5. Sorting Barang
6. Cari Barang
7. Logout

Pilihan:
```

Gambar 4.3 Output Menu Utama

```
--- Tambah Barang ---
Kode Barang: EL004
Nama Barang: Kipas
Kategori: Alat Rumah Tangga
Jumlah: 12
Harga: 500000

--- Lokasi Penyimpanan ---
Gedung: Gudang C
Lantai: 1
Rak: 01

Barang berhasil ditambahkan!
Tekan Enter untuk melanjutkan...
```

Gambar 4.4 Output Tambah Barang

```
==================================================
   DAFTAR INVENTARIS GUDANG
==================================================
+-------+----------------+-------------------+--------+------------+---------------+----------+
| Kode |   Nama Barang  |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus    | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL002 | Mouse Logitech | Aksesoris         | 50     | Rp 150000  | Gudang A-1-02 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL003 | Monitor LG     | Komputer          | 0      | Rp 2000000 | Gudang B-2-05 | Habis    |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas          | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.5 Output Lihat Barang

```
Masukkan kode barang: EL003

Nama Barang baru: Kulkas
Kategori baru: Alat Rumah Tangga
Jumlah baru: 15
Harga baru: 1200000
Gedung baru: Gudang C
Lantai baru: 1
Rak baru: 02

Data berhasil diubah.
Tekan Enter untuk melanjutkan...
```

Gambar 4.6 Output Ubah Data Barang

```
==================================================
   DAFTAR INVENTARIS GUDANG
==================================================
+-------+----------------+-------------------+--------+------------+---------------+----------+
|  Kode |   Nama Barang  |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus    | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL002 | Mouse Logitech | Aksesoris         | 50     | Rp 150000  | Gudang A-1-02 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas         | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas          | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.7 Output Setelah Ubah Data Barang

```
Masukkan kode barang: EL002
Yakin ingin menghapus? (y/n): y

Barang berhasil dihapus!
Tekan Enter untuk melanjutkan...
```

Gambar 4.8 Output Hapus Barang

```
==================================================
   DAFTAR INVENTARIS GUDANG
==================================================
+-------+-------------+-------------------+--------+------------+---------------+----------+
|  Kode | Nama Barang |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas      | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas       | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.9 Output Setelah Hapus Barang

```
==================================================
   MENU SORTING INVENTARIS
==================================================
1. Urutkan Nama Barang (Z ke A)
2. Urutkan Harga (Terkecil ke Terbesar)
3. Urutkan Jumlah Stok (Sedikit ke Banyak)
4. Kembali ke Menu Utama

Pilihan:
```

Gambar 4.10 Output Menu Sorting

```
==================================================
   HASIL SORTING: NAMA BARANG (DESCENDING)
==================================================
+-------+-------------+-------------------+--------+------------+---------------+----------+
|  Kode | Nama Barang |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas      | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas       | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.11 Output Sorting Nama(Descending)

```
==================================================
   HASIL SORTING: HARGA (ASCENDING)
==================================================
+-------+-------------+-------------------+--------+------------+---------------+----------+
|  Kode | Nama Barang |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas       | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas      | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.12 Output Sorting Harga(Ascending)

```
==================================================
   HASIL SORTING: JUMLAH STOK (ASCENDING)
==================================================
+-------+-------------+-------------------+--------+------------+---------------+----------+
|  Kode | Nama Barang |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas       | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas      | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.12 Output Sorting Jumlah Stok(Ascending)

```
==================================================
   MENU PENCARIAN (SEARCHING)
==================================================
1. Cari berdasarkan Nama Barang
2. Cari berdasarkan Harga
3. Kembali ke Menu Utama

Pilihan:
```

Gambar 4.13 Output Menu Searching

```
Masukkan Nama Barang: Laptop Asus

Barang ditemukan!
+-------+-------------+----------+--------+------------+---------------+----------+
| Kode  | Nama Barang | Kategori | Jumlah | Harga      | Lokasi        | Status   |
+-------+-------------+----------+--------+------------+---------------+----------+
| EL001 | Laptop Asus | Komputer | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+-------------+----------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.14 Output Searching Nama

```
Masukkan Harga Barang (Rp): 150000

Barang ditemukan dengan harga Rp 150000!
+-------+----------------+-----------+--------+-----------+---------------+----------+
| Kode  | Nama Barang    | Kategori  | Jumlah | Harga     | Lokasi        | Status   |
+-------+----------------+-----------+--------+-----------+---------------+----------+
| EL002 | Mouse Logitech | Aksesoris | 50     | Rp 150000 | Gudang A-1-02 | Tersedia |
+-------+----------------+-----------+--------+-----------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.15 Output Searching Harga

```
==================================================
   SISTEM MANAJEMEN INVENTARIS GUDANG ELEKTRONIK
==================================================
1. Login
2. Keluar Program

Pilihan: 2

Terima kasih telah menggunakan program ini.
```

Gambar 4.16 Output Keluar dari Program

5. Langkah-langkah GIT

5.1 GIT Init

Digunakan untuk membuat repository Git baru di folder lokal, sehingga folder tersebut siap melacak semua perubahan yang terjadi pada file di dalamnya.

```
PS C:\praktikum-apl\post-test> git init
Initialized empty Git repository in C:/praktikum-apl/post-test/.git/
```

Gambar 5.1 Git Init

5.2 GIT Add

Digunakan untuk menandai perubahan file tertentu agar masuk ke daftar antrean persiapan sebelum disimpan.

```
PS C:\praktikum-apl\post-test> git add .
```

Gambar 5.2 Git Add

5.3 GIT Commit

Digunakan untuk menyimpan semua perubahan yang ada di daftar antrean ke dalam riwayat proyek secara permanen beserta pesan penjelasannya.

```
PS C:\praktikum-apl\post-test\post-test-apl-7> git commit -m 'upload cpp dan exe'
[main 07ad650] upload cpp dan exe
 3 files changed, 491 insertions(+)
 create mode 100644 post-test/post-test-apl-7/2509106008-PirloSyabilaHafuza-PT-7.cpp
 create mode 100644 post-test/post-test-apl-7/2509106008-PirloSyabilaHafuza-PT-7.exe
 create mode 100644 post-test/post-test-apl-7/ErrorHandling.h
```

Gambar 5.3 Git Commit

5.4 GIT Remote

Digunakan untuk menghubungkan repository lokal dengan alamat repository online di GitHub sebagai tujuan penyimpanan.

```
PS C:\praktikum-apl\post-test> git remote add origin https://github.com/limitless1fever/praktikum-apl.git
```

Gambar 5.4 Git Remote

5.5 GIT Push

Digunakan untuk mengirimkan seluruh riwayat perubahan yang telah disimpan dari komputer lokal ke repository online di GitHub.

```
PS C:\praktikum-apl\post-test\post-test-apl-7> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 830.82 KiB | 3.04 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/limitless1fever/praktikum-apl
   c522bde..07ad650  main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.5 Git Push